

Mandag d. 18. august

Opgave 1

Lav tre klasser i en generaliseringsstruktur, der repræsenterer begreberne person, mekaniker og værkfører (på et mekanikerværksted).

For personer registreres navn og adresse. Det skal være muligt både at opdatere og hente navn og adresse.

For mekanikere registreres desuden år for svendeprøve samt en timelønssats. Igen skal det være muligt at hente og opdatere begge attributter.

For værkførere (der også er mekanikere) registreres år for udnævnelse til værkfører samt størrelsen af det tillæg pr. uge, som værkføreren gives ud over den almindelige timeløn. Man skal kunne oprette forekomster af alle tre klasser.

- Lav et klasse-diagram indeholdende de tre klasser og deres indbyrdes sammenhæng
- Programmer klasserne i henhold til klasse-diagrammet

Opgave 2

Udvid klasserne fra opgave 1, så det nu bliver muligt at beregne **ugeløn** for mekanikere og værkførere (en metode `beregnLoen`). Det kan antages at både mekanikere og værkfører har en arbejdsuge på 37 timer.

Er det muligt at definere `beregnLoen` metoden en gang for alle i mekaniker-klassen, eller er det nødvendigt at redefinere metoden i værkførerklassen?

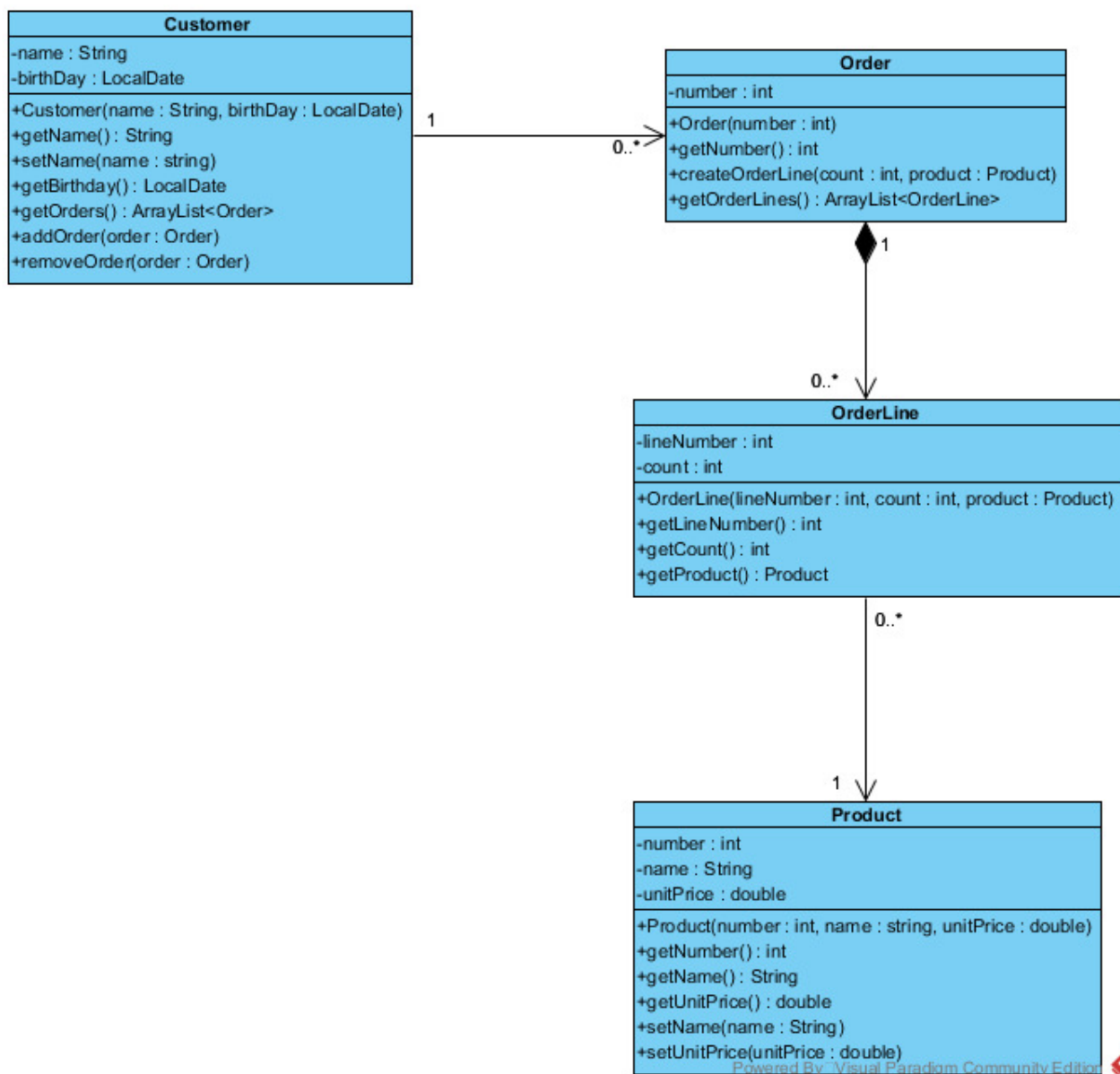
Generaliseringsstrukturen skal gøre det muligt at skrive følgende operation i en App klasse:

```
/**
 * Beregner summen af ugelønnen for alle i listen
 */
public static double samletLoen(ArrayList<Mekaniker> list){...};
```

- Programmer udvidelsen til klasserne, for at håndtere `beregnLoen`.
- Lav en klasse `App` og programmer metoden `samletLoen`
- Opret i anvendelses klassen et antal mekanikere og værkførere, samt indsæt dem i en `ArrayList<Mekaniker>`. Beregn dernæst den samlede ugeløn for alle i listen.
- En synsmand er også ansat på et værksted og er mekaniker af uddannelse. Tilføj klassen `Synsmand` til klassehierarkiet, idet der for hver `Synsmand` holdes rede på, hvor mange syn han har lavet på en uge. Udover den almindelige mekaniker løn får synsmanden et tillæg afhængig af hvor mange syn han har lavet. Tillægget er antal syn i den pågældende uge * 29 kr.
- Tilføj til listen fra c) en `Synsmand` og beregn igen den samlede ugeløn for alle i listen.

Opgave 3

Nedenstående klasse-diagram beskriver et simpelt ordresystem. Koden svarende til systemet kan findes i filen MiniOrderSystem.zip.

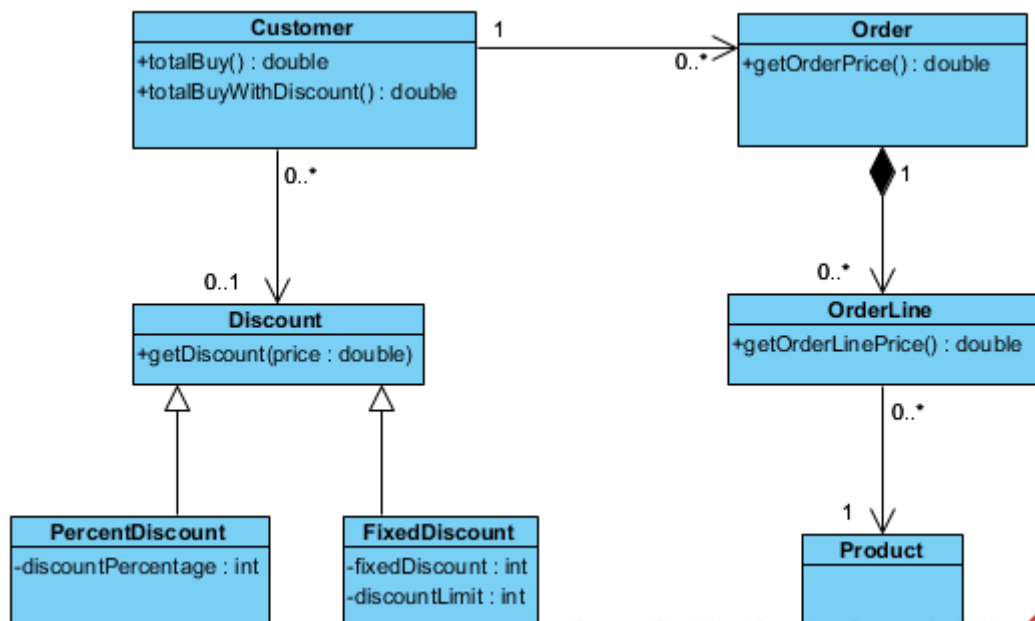


a. Programmér en afprøvningsklasse der opretter:

- 5 Products
- 2 Customers

Den første Customer skal have to Orders tilknyttet, og den anden skal have fire. Hver Order skal indeholde mindst to OrderLines.

- b. Det skal være muligt at beregne den samlede pris for alle de ordrer en kunde har. Der skal derfor tilføjes følgende metoder til modellen:
- `getOrderLinePrice` til klassen `OrderLine` der beregner prisen for ordrelinjen
 - `getOrderPrice` på til klassen `Order` der beregner ordrens pris (summen af priserne for ordrelinjerne)
 - `totalBuy` til klassen `Customer` der beregner den samlede pris for alle de ordrer en kunde har.
- c. Kunderne i `MiniOrderSystem` skal have mulighed for at få forskellige former for rabat. Udvid derfor koden med en `Discount`-klasse og to klasser der nedarver fra `Discount`-klassen som vist nedenfor.



Powered By Visual Paradigm Community Edition

Diagrammet skal læses sådan, at en kunde kan have et `Discount`-objekt knyttet til sig. Metoden `getDiscount(double price)` svarer på, hvor meget en kunde skal have i rabat, når der er købt for beløbet angivet i parameteren `price`. `Discount`-objekt er så aktuelt enten null (ingen rabat), af type `PercentDiscount` (her ydes % rabat af prisen) eller `FixedDiscount` (her ydes rabat hvis beløb er over en vis grænse). I alle tilfælde skal de forskellige `Discount`-typer have en passende constructor der initialiserer `Discount`-objektet med den nødvendige information.

Derudover skal en `Customer` nu svare på, hvad dens samlede pris er, når der gives en samlet rabat, på alle de ordrer en kunde har (metoden `totalBuyWithDiscount`).

- d. Udvid anvendelsesklassen fra spørgsmål a) så første kunde får en `PercentDiscount` på 15% og næste kunde får en fast rabat på 250 kroner, når han har ordre for mere end 1000 kroner. Ret eventuelt en af ordrene, så du er sikker på den samlet har en pris på over 1000 kroner.
- Udskriv priserne for kundernes ordrer både med og uden rabat.